Classes II

Lukas Holik

lukasholik@cs.aau.dk

(based on slides of Gabriela Montoya)

Classes

- Templates used to create objects
- Define the **properties** (fields) and **behavior** (methods) of the objects
- Each object has a copy of all the instance fields (variables)
- Some fields and methods can be **static**
- **new** creates an instance of the class and constructors are used to initialize it
- **this** is used to refer to the current instance
- In Java, parameters are passed **by value** (primitive types) and **by reference value** (reference types) to methods and constructors

Fully qualified name: **java.lang.Object**

Object

- All Java classes extend the Object class (directly or indirectly)
- Object is the **supertype** (superclass) of **all** classes
- A reference variable with type Object can hold any reference (value)

All reference values can be assigned to an object reference

```
Catalog c1 = new Catalog(3);
Object o = c1;
```

A **cast** allows for assigning o to a reference value of type Catalog if possible, otherwise a java.lang.ClassCastException is thrown

```
Catalog c2 = (Catalog) o;
```

Use **instanceof** to check that o holds a reference to an instance of Catalog

```
if (o instanceof Catalog) {
    Catalog c2 = (Catalog) o;
}
```

notify(), notifyAll(), wait() are used for thread synchronization (covered in the 4th semester)

Object's methods

- Methods with an implementation in Object that cannot be reimplemented (cannot be overridden): getClass(), notify(), notifyAll(), wait()
- Methods with an implementation in Object but that can be customized: toString(), equals(), hashCode()
- Methods without an implementation in Object: clone(), finalize()

Add **@Override** when providing a customized implementation (the compiles checks it / identifies unintended overloading)

finalize() is used by the garbage collector to perform cleanup when the object is no longer referenced

```
@Deprecated(since="9")
protected void finalize() throws Throwable
```

```
public final Class<?> getClass()
```

getClass()

- Returns the instance of java.lang.Class that represents the class of the object

```
Catalog courses = new Catalog(3);
Class c = courses.getClass();
System.out.println("getName: "+c.getName());
System.out.println("toString: "+c.toString());
java.lang.reflect.Constructor[] constructors = c.getConstructors();
System.out.println("number of constructors: "+constructors.length);
System.out.println("first constructor: "+constructors[0].toString());
```

```
C:\Users\gmontoya\Documents\OOP\E23\programs>java -cp class catalogExample3
getName: Catalog
toString: class Catalog
number of constructors: 1
first constructor: public Catalog(int)
```

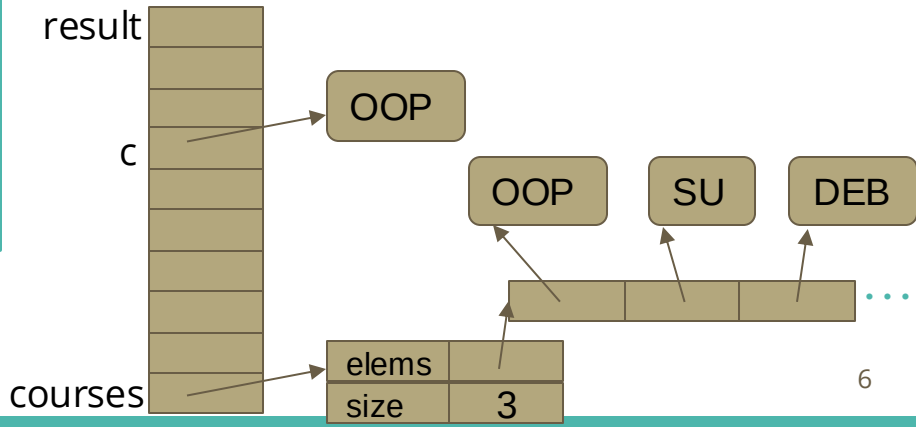
o1 == o2 is true iff o1 and o2 **refer** to the same object

catalogExample2

```
public class catalogExample2 {

      public static void main(String[] args) {

            Catalog courses = new Catalog(45);

            courses.addElem("OOP");
            courses.addElem("SU");
            courses.addElem("DEB"); ...

            String c = args[0]; ...
            String result = courses.find(c);
            System.out.println(result);
      }
}
```

```
public String find(String e) {
    String res;
    for (String currentElem : this.elems) {
        if (currentElem==e) {
            res = currentElem;
        }
    }
    return res;
}
```

```
java -cp class catalogExample2 OOP
```



```
public boolean equals(Object obj)
```

equals(Object)

- Returns true if and only if the object passed as parameter “is the same” as **this**

```
boolean sameObject =  o1.equals(o2);
```

By **default**, it uses ==.
`o1.equals(o2)` iff o1 and o2 have the same address (reference, are the same instance)

It is often redefined to use other criteria, e.g. String - comparison of the actual sequences of characters.

Requirements of the reimplementation:

- **reflexive**, **symmetric**, **transitive**
- Consistent
- An instance cannot be equal to null
- It must be consistent with hashCode

catalogExample2

equals

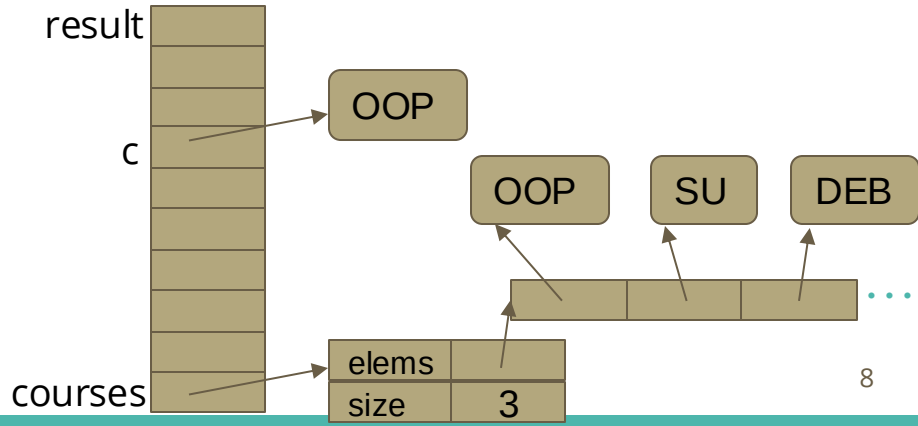
`public boolean equals(Object anObject)`

Compares this string to the specified object. The result is `true` if and only if the argument is not `null` and is a `String` object that represents the same sequence of characters as this object.

```
public String find(String e) {
    String res;
    for (String currentElem : this.elems) {
        if (currentElem.equals(e)) {
            res = currentElem;
        }
    }
    return res;
}
```

o1.equals(o2) is true iff o1 and o2 *represent* the same object

```
java -cp class catalogExample2 OOP
```



```
public int hashCode()
```

hashCode()

- A hash code is an integer value that is computed for a piece of information using an algorithm.

```
public class Catalog {
    private String[] elems = null;
    private int size = 0;

    public Catalog(int maxSize) {
        this.elems = new String[maxSize];
    }
    public void addElem(String e) {
        this.elems[size] = e;
        this.size++;
    }
    ...
}
```

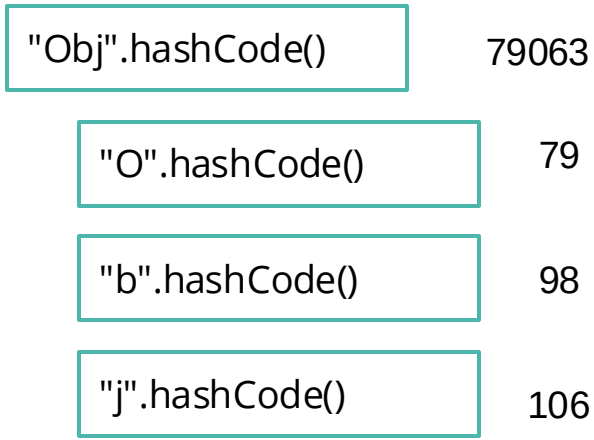
```
jshell> Catalog courses = new Catalog(3);
courses ==> Catalog@736e9adb

jshell> System.out.println(courses.hashCode());
1936628443
```

The **default implementation** converts the memory address of courses into an integer.

```
public int hashCode()
```

hashCode()

- Reimplementations typically rely on the instance fields of the class



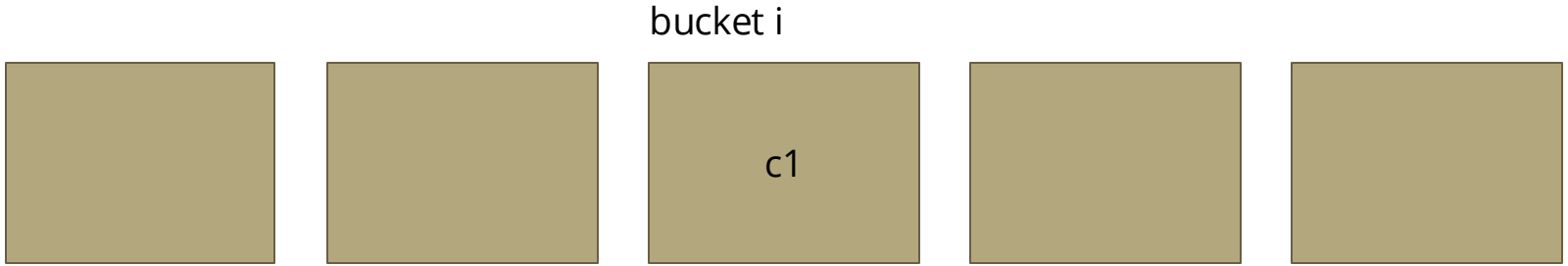
79*31^2 + 98*31^1 + 106*31^0

75919 + 3038 + 106

79063

Java uses hash codes to efficiently retrieve data from **hash-based collections.**

How are hash codes typically used?



We want to store courses into *buckets.*

- Where is the course c1 stored?
- How do we know if c1 is in one of the buckets?

```
int i = c1.hashCode();
```

But often there are **less buckets** than hash code values (integers)

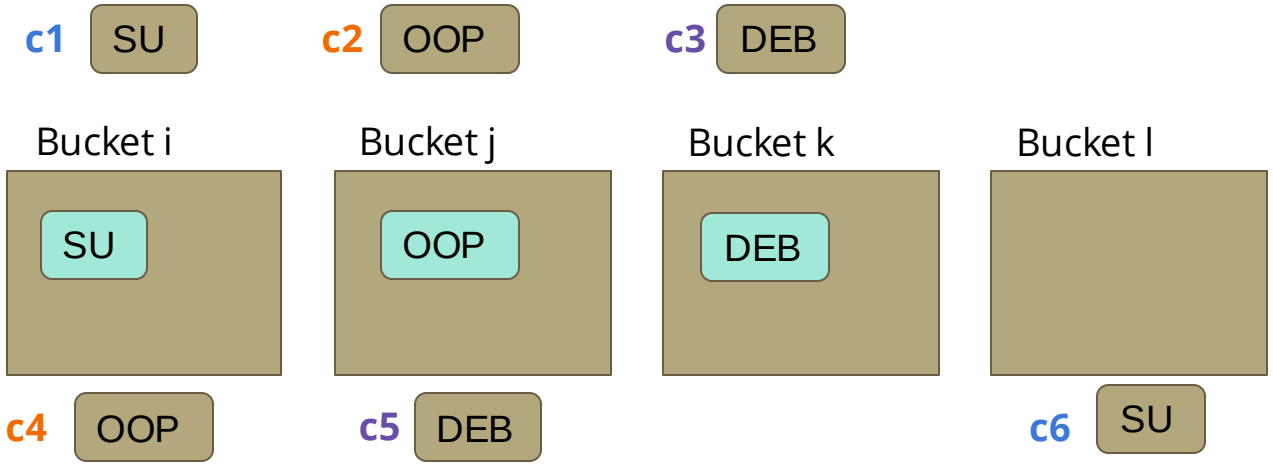
the hash code:

- Two different objects can have the same hash code.
- Multiple calls on the same object must produce the same int value.
- If o1 and o2 *represent* the same object, they must have the same hash code.
- It can be negative.

hashCode()

(a) insert one element multiple times
(b) failing to retrieve existing elements
(c) none

- The default implementation obtains an integer from the memory address of the object



c1.hashCode() i
c2.hashCode() j
c3.hashCode() k
c4.hashCode() i
c5.hashCode() j
c6.hashCode() l

Problems using the default implementation for objects stored in a java.util.Hash**Set**?

When are two Courses equal?

```
import java.util.ArrayList;

class Course {

    private String code;

    private String title;

    private ArrayList<String> goals;

    ...

}
```

When are two instances of this class representing the same course?

- Two instances with the same code must be the same course?
- Two instances with the same code, title, and learning goals must be the same course?

Both can be good choices depending on the domain, but… if we redefine equals() or hashCode(), we must consistently redefine the other one

Is this a suitable reimplementation of equals(Object)?

```
import java.util.ArrayList;

class Course {

    private String code;

    private String title;

    private ArrayList<String> goals;

    ...

}
```

```
public boolean equals(Object o) {
    return this.code.equals(o.code);
}
```

(a) Always
(b) Never
(c) Sometimes

- Can we pass null as actual parameter?
- Can we pass an instance of Catalog as actual parameter?

equals(Object) should:

1. Be **reflexive, transitive, symmetric**.
2. Return **false** on comparison with **null**.
3. Be **consistent**: x.equals(y) returns the same result unless x or y is modified.
4. Be **consistent with hashCode**: if x.equals(y) then x.hashCode() == y.hashCode().
5. Return **false** on comparison with an **incompatible type**.

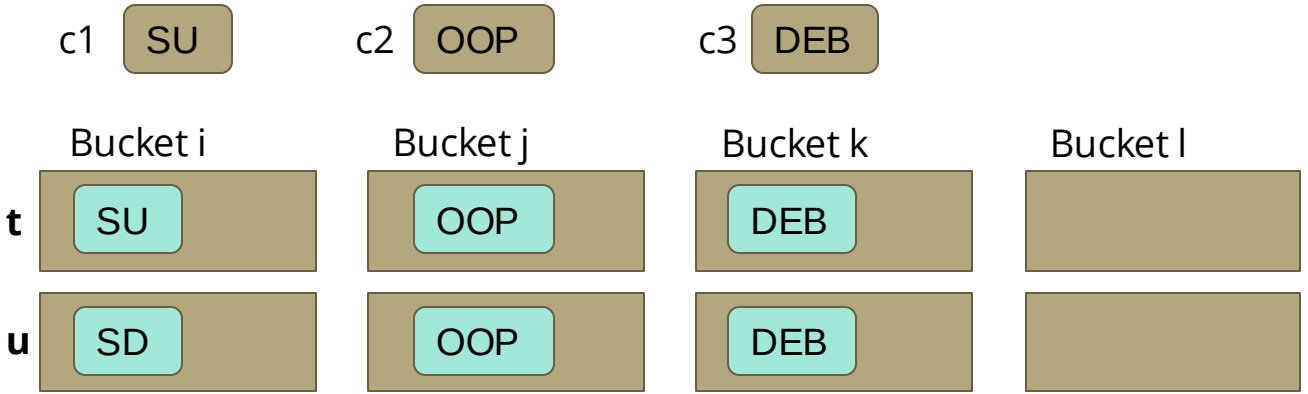
```
import java.util.ArrayList;

public class Course {
    private String code;
    private String title;
    private ArrayList<String> goals;
    ...
    public boolean equals(Object other) {
        if ((other == null) ||
            !(other instanceof Course)) {
            return false;
        }
        Course c2 = (Course) other;
        return this.code.equals(c2.code);
    }
}
```

Two courses are the same if they have the same code...

```
HashSet<Course> cs = new HashSet<Course>();
cs.add(c1);
cs.add(c2);
cs.add(c3);
// time t
c1.setCode("SD");
// time u
System.out.println(cs.contains(c1));
```

```
public int hashCode() {
    return code.hashCode();
}
```

Is **c1** part of the collection cs (represented by buckets i, j, k, l) at **time u**?



	time t	time u
c1.hashCode()	i	k
c2.hashCode()	j	j
c3.hashCode()	k	k

toString()

```
public String toString()
```

- How to represent an object in a string form
- Useful for debugging

Used for String concatenation (+) and when a reference is passed to `System.out.print()` or `System.out.println()`

```
Catalog c1 = new Catalog(3);
```

Default implementation

```
Catalog@6477463f
```

Class name

Hash code in hexadecimal

reimplementations typically rely on the instance fields of the class

```
Catalog ( capacity: 3; elems: < >
)
```

toString()

reimplementations typically rely on the instance fields of the class

```
public class Catalog {
    private String[] elems = null;
    private int size = 0;
    ...
    @Override
    public String toString() {
        int capacity = elems.length;
        String str = "Catalog ( capacity: " + capacity
                     + "; elems: <";
        for (int i = 0; i < this.size; i++) {
            str += this.elems[i];
            if (i < size - 1) {
                str += ", ";
            }
        }
        str += "> )";
        return str;
    }
}
```

```
Catalog c1 = new Catalog(3);
c1.addElem("SU");
c1.addElem("OOP");
String str1 = c1.toString();
```

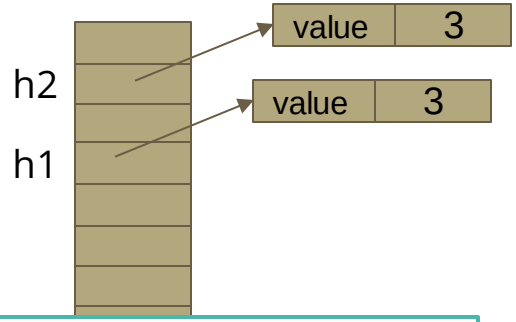
```
Catalog ( capacity: 3; elems: <SU,OOP> )
```

```
protected Object clone() throws CloneNotSupportedException
```

clone

- The implementation has protected access (client code cannot use it directly)
- The implementation in Object is a **bitwise copy** of the original object
- Classes that implement clone() must include “implements Cloneable”

```
class Holder implements Cloneable {
    private int value;
    public Holder(int v) {
        value = v;
    }
    ...
    public void setValue(int v) {
        this.value = v;
    }
}
```




```
Holder h1 = new Holder(3);
Holder h2 = h1.clone();
```

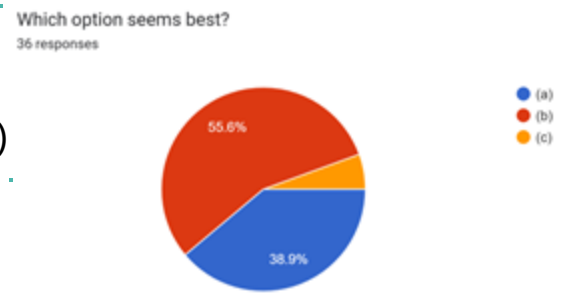
```
h2.setValue(5);
```

Value of h1.value?

(a) 3
(b) 5
(c) No value

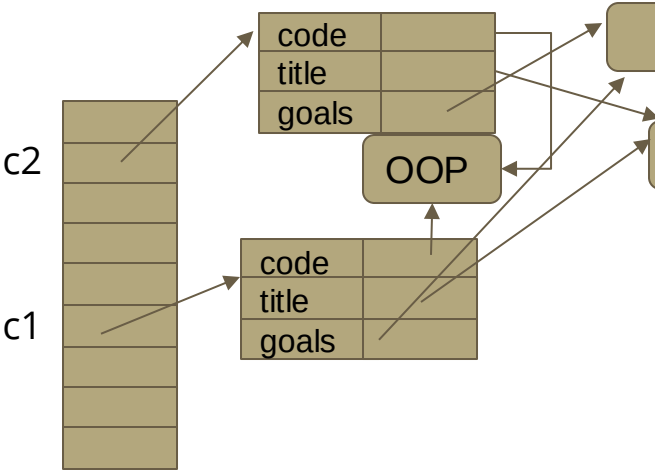
clone

For classes with instance variables of only primitive types or String, the default is enough
(Strings cannot be changed after creation,they are *immutable*)



- Default implementation corresponds to a **shallow copy** and sometimes is not what is appropriate for the objects

```
class Course implements Cloneable {
  private String code;
  private String title;
  private ArrayList<String> goals;
  ...
  public void addGoal(String g) {
      goals.add(g);
  }
  public void removeLastGoal() {
      goals.remove(goals.size()-1);
  }
}
```



- apply correct terminology

Object-oriented programming

```
c2.removeLastGoal();
```

How many goals does **c1** have?

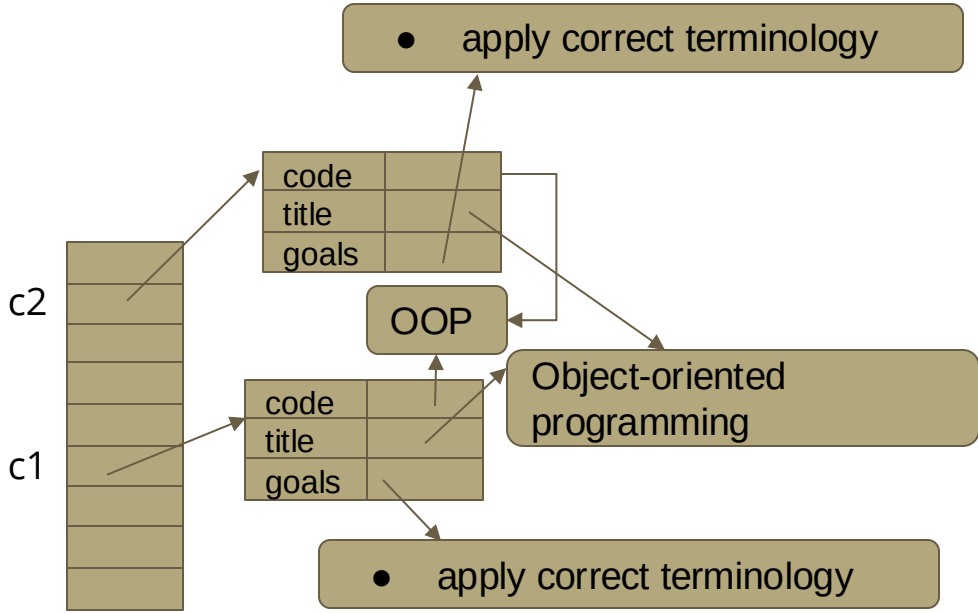
(a) 0
(b) 1
(c) 2

```
Course c1 = new Course("OOP", "Object-oriented programming");
c1.addGoal("apply correct terminology");
Course c2 = (Course) c1.clone();
```

How do we obtain a deeper copy of Course?

```
import java.util.ArrayList;

class Course implements Cloneable {
  private String code;
  private String title;
  private ArrayList<String> goals;
  public Course (String c, String t) {
      this.code = c;
      this.title = t;
      this.goals = new ArrayList<String>();
  }
  public void addGoal(String g) {
      goals.add(g);
  }
  ...
  public Course clone() {
     Course c = new Course(code, title);
     for (String g : goals) {
         c.addGoal(g);
     }
     return c;
  }
}
```

```
Course c1 = new Course("OOP", "Object-oriented programming");
c1.addGoal("apply correct terminology");
Course c2 = c1.clone();
```



How do we make Course an immutable class?

```
class Course {
  private String code;
  private String title;
  private ArrayList<String> goals;
  public Course (String c, String t, ArrayList<String> gs) {
      this.code = c;
      this.title = t;
      this.goals = new ArrayList<String>();
      for (String g : gs) {
          this.goals.add(g);
      }
  }
  public String getCode() {
      return this.code;
  }
  public String getTitle() {
      return this.title;
  }
  public ArrayList<String> getGoals() {
      return this.goals.clone();
  }
}
```

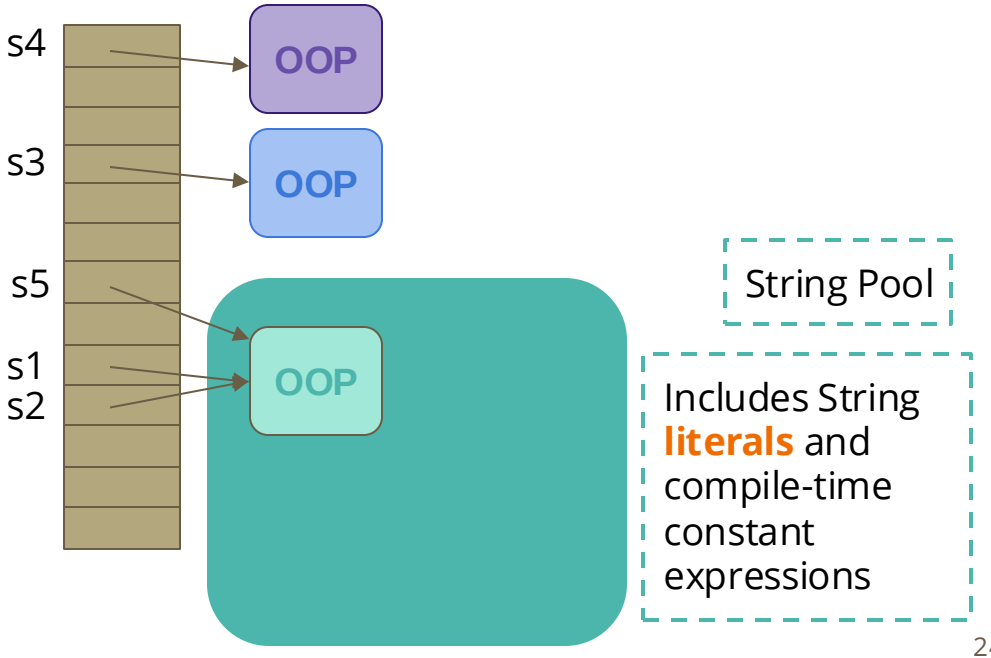
- After an instance of Course is created, it cannot be changed
- None of the methods allows for directly or indirectly changing the state of the object
- A Course object is both an *internally* and an *externally* immutable object

https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/String.html

String Pool

- **Set** of Strings maintained by the JVM to improve performance and minimize memory usage

```
String s1 = "OOP";
String s2 = "OOP";
String s3 = new String("OOP");
String s4 = new String("OOP");
String s5 = "O"+"O"+"P";
```



https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/String.html

String Pool

intern

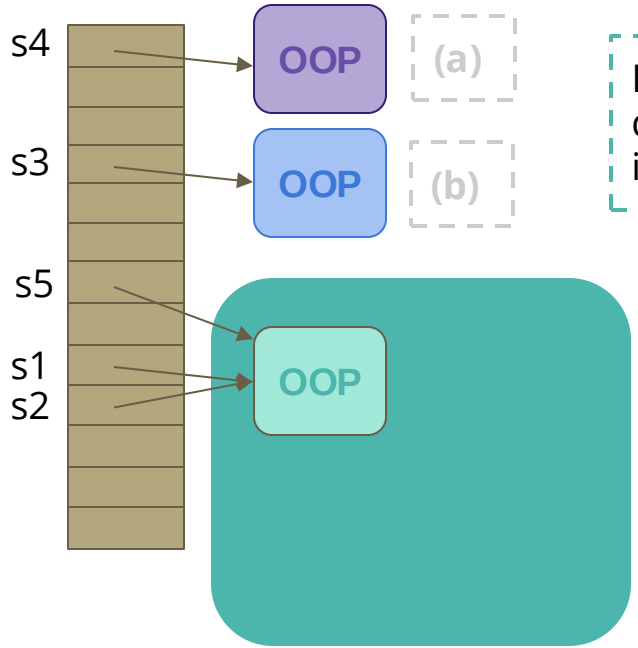
`public String intern()`

Returns a canonical representation for the string object.

A pool of strings, initially empty, is maintained privately by the class `String`.

When the intern method is invoked, if the pool already contains a string equal to this `String` object as determined by the `equals(Object)` method, then the string from the pool is returned. Otherwise, this `String` object is added to the pool and a reference to this `String` object is returned.

```
String s1 = "OOP";
String s2 = "OOP";
String s3 = new String("OOP");
String s4 = new String("OOP");
String s5 = "OOP";
String s6 = s4.intern();
```



In most cases, programmers do not need to create a new instance of String

Which object is referred by **s6**?

How can a String be efficiently retrieved from this pool?

String Pool

https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/String.html

Identify if a String is palindrome

"A palindrome is a word, a verse, a sentence, or a number that reads the same in forward and backward directions"

noon

```
public static boolean isPalindrome(String str) {

    boolean isPalindrome = true;

    for (int i = 0; i < str.length() / 2 ; i++) {
        if ( str.charAt(i) != str.charAt(str.length() - i-1) ) {
            isPalindrome = false;
        }
    }
    return isPalindrome;
}
```

```
jshell> isPalindrome("noon");
$9 ==> true

jshell> isPalindrome("morning");
$10 ==> false
```

charAt

```
public char charAt(int index)
```

Returns the `char` value at the specified index. An index ranges from `0` to `length() - 1`. The first `char` value of the sequence is at index `0`, the next at index `1`, and so on, as for array indexing.

getChars

```
public void getChars(int srcBegin,
                     int srcEnd,
                     char[] dst,
                     int dstBegin)
```

Copies characters from this string into the destination character array.

The first character to be copied is at index `srcBegin`; the last character to be copied is at index `srcEnd-1` (thus the total number of characters to be copied is `srcEnd-srcBegin`). The characters are copied into the subarray of `dst` starting at index `dstBegin` and ending at index:

```
dstBegin + (srcEnd-srcBegin) - 1
```

length

```
public int length()
```

Returns the length of this string. The length is equal to the number of Unicode code units in the string.

If the `char` value specified at the given index is in the high-surrogate range, the following index is less than the length of this `String`, and the `char` value at the following index is in the

java.lang.String

java.lang.Character

Check if a String is a valid password

```
class User {
    private String name;
    ...
    public User(String n) {
        this.name = n;
    }
    public String getName() {
        return this.name;
    }
}
```

```
public static void main(String[] args) {
    String name = args[0];
    String password = args[1];
    User u = new User(name);
    System.out.println(password+" is "
       +(isValidPassword(u, password)?"":"not ")
           +"a valid password for user "+name);
}
```

```
C:\Users\gmontoya\Documents\OOP\E23\programs>java -cp class testStringFunctions Jens Jens2353!2340
Jens2353!2340 is not a valid password for user Jens

C:\Users\gmontoya\Documents\OOP\E23\programs>java -cp class testStringFunctions Jens AAUDK2353!2340
AAUDK2353!2340 is not a valid password for user Jens

C:\Users\gmontoya\Documents\OOP\E23\programs>java -cp class testStringFunctions Jens AAUjnsDK235!40
AAUjnsDK235!40 is a valid password for user Jens
```

This is the solution proposed during the lecture but to cover the first example, **?1** needs to consider more than exact matches. An alternative solution is in the appendix

Check if a String is a valid password

```
public static boolean isValidPassword(User u, String p) {
    if ( u.getName().equals(p) ) {
        return false;
    }
    int numDigits = 0;
    int numLowerCaseLetters = 0;
    int numUpperCaseLetters = 0;
    int numOthers = 0;
    for (int i = 0; i < p.length(); i++) {
        char c = p.charAt(i);
        switch (Character.getType(c)) {
            case Character.UPPERCASE_LETTER:
                numUpperCaseLetters++;
                break;
            case Character.LOWERCASE_LETTER:
                numLowerCaseLetters++;
                break;
            case Character.DECIMAL_DIGIT_NUMBER:
                numDigits++;
                break;
            default:
                numOthers++;
        }
    }
    return numDigits > 0 && numLowerCaseLetters > 0
         && numUpperCaseLetters > 0 && numOthers > 0;
}
```

```
class User {
    private String name;
    ...
    public User(String n) {
        this.name = n;
    }
    public String getName() {
        return this.name;
    }
}
```

java.lang.String

charAt

```
public char charAt(int index)
```

Returns the char value at the specified index. An index ranges from 0 to length() - 1. The first char value of the sequence is at index 0, the next at index 1, and so on, as for array indexing.

getChars

```
public void getChars(int srcBegin,
                     int srcEnd,
                     char[] dst,
                     int dstBegin)
```

Copies characters from this string into the destination character array.

The first character to be copied is at index srcBegin; the last character to be copied is at index srcEnd-1 (thus the total number of characters to be copied is srcEnd-srcBegin). The characters are copied into the subarray of dst starting at index dstBegin and ending at index:

```
dstBegin + (srcEnd-srcBegin) - 1
```

length

```
public int length()
```

Returns the length of this string. The length is equal to the number of Unicode code units in the string.

If the char value specified at the given index is in the high-surrogate range, the following

java.lang.Character

isLowerCase

```
public static boolean isLowerCase(char ch)
```

Determines if the specified character is a lowercase character.

isUpperCase

```
public static boolean isUpperCase(char ch)
```

Determines if the specified character is an uppercase character.

isDigit

```
public static boolean isDigit(char ch)
```

Determines if the specified character is a digit.

isLetter

```
public static boolean isLetter(int codePoint)
```

Determines if the specified character (Unicode code point) is a letter.

getType

```
public static int getType(char ch)
```

Returns a value indicating a character's general category.

```
C:\Users\gmontoya\Documents\OOP\E23\programs>java -cp class testStringFunctions Jens Jens2353!2340
Jens2353!2340 is not a valid password for user Jens

C:\Users\gmontoya\Documents\OOP\E23\programs>java -cp class testStringFunctions Jens AAUDK2353!2340
AAUDK2353!2340 is not a valid password for user Jens

C:\Users\gmontoya\Documents\OOP\E23\programs>java -cp class testStringFunctions Jens AAUjnsDK235!40
AAUjnsDK235!40 is a valid password for user Jens
```

Check if the user is minor

```
import java.time.LocalDate;

class User {
    private String name;
    private LocalDate birthdate;

    public User(String n) {
        this.name = n;
    }
    public String getName() {
        return this.name;
    }
    public void setBirthDate(int year, int month, int day) {
        birthdate = LocalDate.of(year, month, day);
    }
    public LocalDate getBithDate() {
        return birthdate;
    }
    public boolean isMinor() {
        LocalDate t0 = LocalDate.now().minusYears(18);
        return birthdate.compareTo(t0) >= 0;
    }
}
```

```
User u = new User("Jens");

u.setBirthDate(2000, 06, 01);

boolean b1 = u.isMinor();

u.setBirthDate(2010, 06, 01);

boolean b2 = u.isMinor();
```

The value of **b1** must be **false** and the value of **b2** must be **true**

java.util.Objects

Objects

- Utility class with 20 static methods that can be used for bounds checks, comparing objects, computing hash code, checking for null, validating arguments, obtaining string representation of objects

java.util.Objects

Objects: Bounds checks

`Objects.checkFromToIndex(fromIndex, toIndex, length)` returns `fromIndex` if [`fromIndex`,..., `toIndex`) is within the range [0 .. `length`)

```
int fromIndex = 5, toIndex = 10, length = 20;
int i = Objects.checkFromToIndex(fromIndex, toIndex, length);

fromIndex = 5, toIndex = 10, length = 5;
int j = Objects.checkFromToIndex(fromIndex, toIndex, length);
```

Are values between 5 and 9 included between 0 and 19?

Are values between 5 and 9 included between 0 and 4?

If the condition is not satisfied, an `IndexOutOfBoundsException` is thrown
(exceptions will be covered in a couple of weeks)

Objects: Comparing objects

`Objects.equals(object1, object2)` returns `true` if object1 and object2 are equal (that includes if both are null)

```
int[] a1 = { 1, 2, 3 };
int[] a2 = { 1, 2, 3 };
boolean b = Objects.equals(a1, a2);
```

What is the value of b?

(a) **false**
(b) true
(c) No value

```
int[] a1 = { };
int[] a2 = { };
boolean b = Objects.equals(a1, a2);
```

Objects: Comparing objects

`Objects.deepEquals(object1, object2)` returns `true` if object1 and object2 are deeply equal (that includes if both are null)

If the actual parameters are arrays, they must have the same length and each pair of elements in the same position must be deeply equal

```
int[] a1 = { 1, 2, 3 };
int[] a2 = { 1, 2, 3 };
boolean b = Objects.deepEquals(a1, a2);
```

```
jshell> int[] a1 = { 1, 2, 3};
a1 ==> int[3] { 1, 2, 3 }

jshell> int[] a2 = { 1, 2, 3};
a2 ==> int[3] { 1, 2, 3 }

jshell> boolean b = Objects.deepEquals(a1, a2);
b ==> true
```

Why would anyone use Objects.equals(object1, object2) instead of Objects.deepEquals(object1, object2)?

Wrapper Classes

- Classes used to represent primitive types
- A typical use is converting between types, e.g., from String to integer

```
public static void main(String[] args) {
    int start = (new Integer(args[0])).intValue();
    int end = Integer.parseInt(args[1]);
    int step = Integer.parseInt(args[2]);
    Boolean showProgress = Boolean.valueOf(args[3]);
}
```

Primitive Type	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

Using constructors create new instances, but using valueOf reuses existing instances (for numbers between -128 and + 127)

Constructors have been *deprecated*
They will be removed in future versions and we should avoid using them if possible

Autoboxing and unboxing

- Autoboxing: **automatic** wrapping from a primitive type value as an instance of its wrapper class
- Unboxing: **automatic** unwrapping an object to its primitive type

```
Integer i = 5;
int v = i;
```

With autoboxing/unboxing

```
Integer i = Integer.valueOf(5);
int v = i.intValue();
```

With explicit wrapping/unwrapping

Use of autoboxing/unboxing of actual parameters is only done if no possible match is found

```
ArrayList<Integer> list = new ArrayList<Integer>();
list.add(9);
int i = list.get(0);
```

Classes II

- java.lang.Object is the **supertype** of all Java classes
- equals(), toString(), and hashCode() have a **default implementation based on memory addresses**, but new classes can provide their own implementation
- clone() implemented in the class Object provides a **shallow copy** of the instance
- It is a bad idea to use mutable object as keys in Hash-code based collections
- It is a good idea to reuse existing classes, e.g., java.lang.String, java.time.LocalDate, java.util.ArrayList

Additional Exercises

1. Implement the method equals in the class Catalog.java and consider that two catalogs are equal if their elements are equals (pairwise)
   a. Hint: you could use some methods of the class java.util.Arrays
2. Implement the method hashCode in the class Catalog.java. The implementation must consider the instance variables of the class and be consistent with your implementation of equals (previous task)
   a. Hint: you could use some methods of the class java.util.Arrays
3. Implement the method clone in the class Catalog.java. The implementation must provide a deep copy of the catalog (changes in the cloned catalog do not change the state of the original catalog)
   a. Hint: you could use some methods of the class java.util.Arrays

Appendix

Check if a String is a valid password

```
public static boolean isValidPassword(User u, String p) {
    if ( (p.indexOf(u.getName())>=0)||(p.length() < 12) ) {
        return false;
    }
    int numDigits = 0;
    int numLowerCaseLetters = 0;
    int numUpperCaseLetters = 0;
    int numOthers = 0;
    for (int i = 0; i < p.length(); i++) {
        char c = p.charAt(i);
        switch (Character.getType(c)) {
            case Character.UPPERCASE_LETTER:
                numUpperCaseLetters++;
                break;
            case Character.LOWERCASE_LETTER:
                numLowerCaseLetters++;
                break;
            case Character.DECIMAL_DIGIT_NUMBER:
                numDigits++;
                break;
            default:
                numOthers++;
        }
    }
    return numDigits > 0 && numLowerCaseLetters > 0
         && numUpperCaseLetters > 0 && numOthers > 0;
}
```

```
class User {
    private String name;
    ...
    public User(String n) {
        this.name = n;
    }
    public String getName() {
        return this.name;
    }
}
```

java.lang.String

charAt

```
public char charAt(int index)
```

Returns the char value at the specified index. An index ranges from 0 to length() - 1. The first char value of the sequence is at index 0, the next at index 1, and so on, as for array indexing.

getChars

```
public void getChars(int srcBegin,
                     int srcEnd,
                     char[] dst,
                     int dstBegin)
```

Copies characters from this string into the destination character array.

The first character to be copied is at index srcBegin; the last character to be copied is at index srcEnd-1 (thus the total number of characters to be copied is srcEnd-srcBegin). The characters are copied into the subarray of dst starting at index dstBegin and ending at index:

```
dstBegin + (srcEnd-srcBegin) - 1
```

length

```
public int length()
```

Returns the length of this string. The length is equal to the number of Unicode code units in the string.

If the char value specified at the given index is in the high-surrogate range, the following index is less than the length of this String, and the char value at the following index is in the

java.lang.Character

isLowerCase

```
public static boolean isLowerCase(char ch)
```

Determines if the specified character is a lowercase character.

isUpperCase

```
public static boolean isUpperCase(char ch)
```

Determines if the specified character is an uppercase character.

isDigit

```
public static boolean isDigit(char ch)
```

Determines if the specified character is a digit.

isLetter

```
public static boolean isLetter(int codePoint)
```

Determines if the specified character (Unicode code point) is a letter.

getType

```
public static int getType(char ch)
```

Returns a value indicating a character's general category.

```
C:\Users\gmontoya\Documents\OOP\E23\programs>java -cp class testStringFunctions Jens Jens2353!2340
Jens2353!2340 is not a valid password for user Jens

C:\Users\gmontoya\Documents\OOP\E23\programs>java -cp class testStringFunctions Jens AAUDK2353!2340
AAUDK2353!2340 is not a valid password for user Jens

C:\Users\gmontoya\Documents\OOP\E23\programs>java -cp class testStringFunctions Jens AAUjnsDK235!40
AAUjnsDK235!40 is a valid password for user Jens
```

hashCode()

int range:
From -2,147,483,648 Until + **2,147,483,647**

- A hash code is an integer value that is computed for a piece of information using an algorithm

"Object".hashCode()	-12939501217		
"O".hashCode()	79	"e".hashCode()	101
"b".hashCode()	98	"c".hashCode()	99
"j".hashCode()	106	"t".hashCode()	116

79*31^5 + 98*31^4 + 106*31^3 +
101*31^2 + 99*31^1 + 116*31^0

2,261,702,929 + 90,505,058 +
3,157,846 + 97,061 + 3,069 + 116

2,355,466,079

But this value cannot be represented using an int

No ambiguity is allowed

Only **public** classes can be used from other packages

Accessing classes from other modules

Module **aau.adt**

aau.adt/dk/aau/adt/Catalog.java

```
package dk.aau.adt;

public class Catalog {
    private String[] elems = null;
    private int size = 0;
    private static int count = 0;
    public Catalog(int maxSize) {
        this.elems = new String[maxSize];
        count++;
    }
    public void addElem(String e) {
        this.elems[size] = e;
        this.size++;
    }
    ...
}
```

Module **aau.tests**

aau.tests/dk/aau/tests/CatalogTest.java

```
package dk.aau.tests;

import dk.aau.adt.Catalog;

class CatalogTest {
   public static void main(String args[]) {
       Catalog coursesSW3 = new Catalog(3);
       ...
   }
}
```

Modules that require aau.tests will automatically require aau.adt (due to the use of **transitive**)

```
module aau.tests {
    requires transitive aau.adt;
}
```

aau.tests/module-info.java

```
module aau.adt {
    exports dk.aau.adt;
}
```

aau.adt/module-info.java

The modifier **static** denotes an optional (at runtime) requires

```
module tests {
    requires static aau.tests;
}
```

tests/module-info.java